Independent Research Evaluation and Architectural Critique: AURA (Agent Uncertainty and Residue Arbiter)

You are investigating one of the most consequential systems problems in modern AI engineering: the structural gap between an AI agent invoking a sequence of tools that return local success codes and the agent genuinely fulfilling a user's multi-step request without corrupting the surrounding environment. Recent empirical studies confirm your core intuition—in a July 2026 agent operating system study, agents reported task success in 90% of episodes where they had actually duplicated a non-idempotent effect; in the October 2026 CTXGAP benchmark, agent final reports omitted 48.0% of recoverable execution-state details (a 40.7 percentage point gap compared to full-context inspection); and in the October 2026 RITE benchmark, across 10 LLMs, silent tool data corruption roughly halved task success and went unnoticed in 66.4% to 79.2% of runs (models caught the bad data in only 20.8% to 33.6% of runs, as reported in the study and its technical summary). However, **AURA as currently proposed is a partial reinvention of existing state-diff evaluation frameworks** (specifically AppWorld, WorkBench, and Agent-Diff) and runtime outcome validators (EffectMatch). Furthermore, your proposed experimental comparison (Approach A: Trust Agent Self-Report vs. Approach B: Tool-Level Verification vs. Approach C: AURA Workflow-Level Verification) suffers from a severe **information-asymmetry confound**: if AURA is granted white-box post-execution read access to the SQLite database and filesystem while the baselines only inspect tool return payloads, AURA's higher accuracy is a trivial consequence of privileged observability rather than superior verification logic.

To answer your central foundational question first: **If we were designing a reliable AI-agent task-completion verification system from scratch, without knowing anything about AURA, what would the simplest defensible architecture be, what could it actually guarantee, and how would we scientifically demonstrate its value?** The simplest defensible architecture is a **Dual-Surface Delta-Trace Arbiter** governed by an explicit **Observation Boundary Contract**. Instead of five overlapping qualitative dimensions, it requires three formal verification planes evaluated over a three-valued Strong Kleene logic (\mathcal{K}_3 = \{1, \frac{1}{2}, 0\}, mapping to PASS, UNCERTAIN, and FAIL):

1. **A Canonical State-Diff Engine (\Delta S = \text{Diff}(S_0, S_T))** that snapshots all mutable storage tables and filesystem directories before (S_0) and after (S_T) execution, verifying positive postconditions (including strict record-multiplicity bounds) and enforcing a **Closed-World Frame Invariant** where any diff entry not explicitly accounted for by a positive task assertion automatically triggers a residue failure (\Delta S \setminus \Delta S_{\text{claimed}} = \emptyset).
2. **A Typed Causal Event Trace (\tau)** that records every tool invocation, input argument, returned payload, and state-read witness to enforce **Data-Flow Provenance**—verifying that the values written to the final database and report were causally derived from valid tool reads rather than guessed from parametric memory or corrupted by silent tool errors.
3. **An Epistemic Boundary Evaluator** that outputs UNCERTAIN strictly when a required postcondition or frame surface lies across an unconfirmed or degraded observation

boundary (such as an external API call that timed out without a status endpoint, or a database table lacking readback permissions), rather than using UNCERTAIN as a vague fallback for poorly written contracts.

Within a snapshot-capable sandbox, this scratch-built architecture guarantees **sound and complete persistent state-mutation accounting**, **witness-grounded execution provenance**, and **calibrated epistemic abstention** when effects cross unobservable boundaries. It *cannot* guarantee alignment with unstated user intent (the specification translation bottleneck), nor can state diffing alone detect read-only privacy leaks (exfiltration) unless paired with trace inspection. You scientifically demonstrate its value in two ways: first, by enforcing **information parity** across baselines (giving an agentic self-verifier, a state-diff-only checker, and a trace-only checker the exact same sandbox readback access and contract definitions); and second, by demonstrating via controlled fault and mutation injection that **joint state-diff and trace-provenance arbitration** eliminates the specific blind spots—duplicate non-idempotent writes, silent tool corruption, and negative-space validation omissions—that defeat single-surface verifiers.

1. First-Principles Analysis: What Is the Real Underlying Problem?

To understand what it means for an AI agent to genuinely complete a task, we must strip away LLM-specific terminology and analyze the agent as an untrusted, stochastic controller interacting with a stateful, partially observable distributed environment.

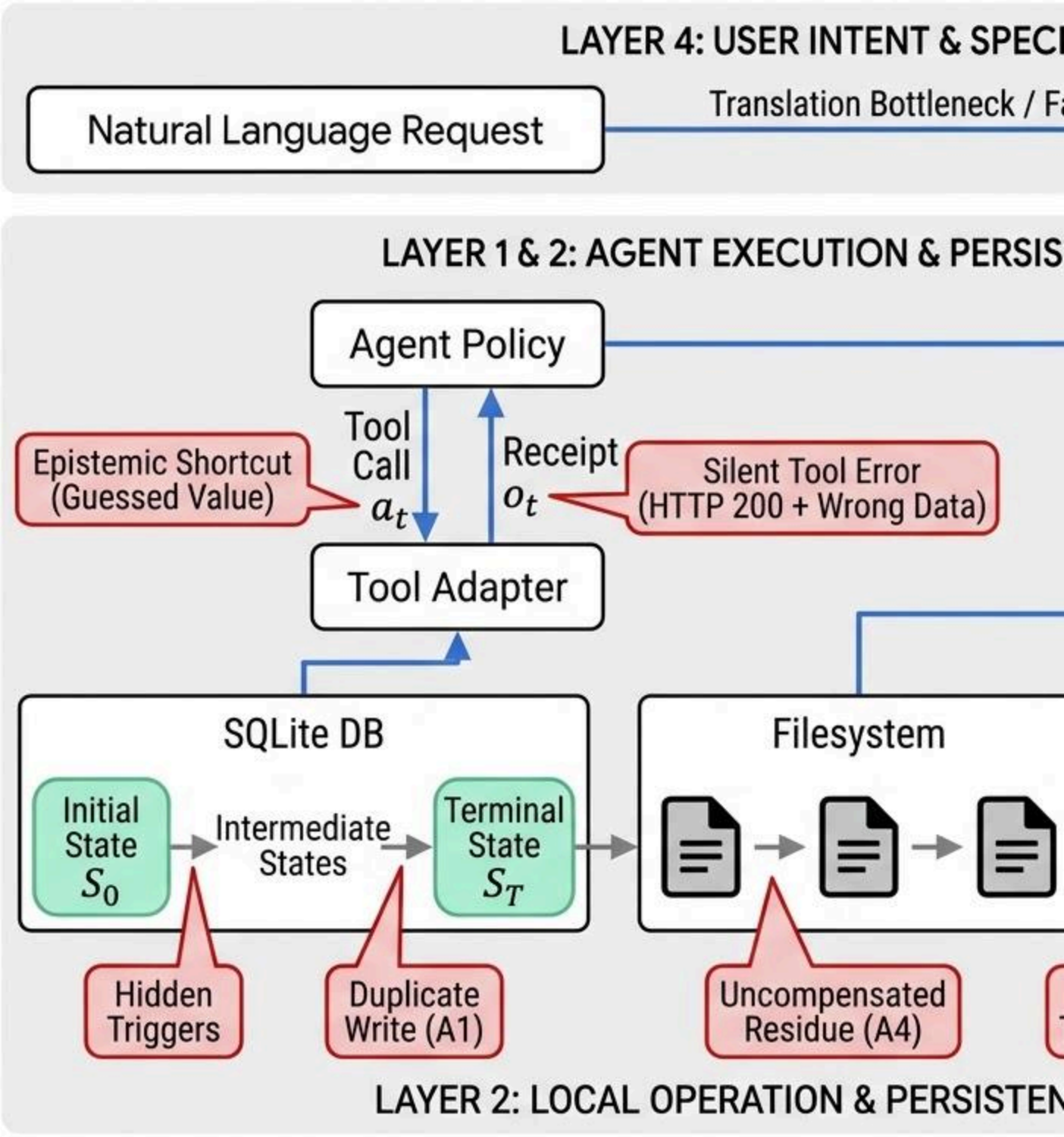
A.1 The Four-Layer Hierarchy of Execution Correctness

Let an agentic environment be a tuple \mathcal{E} = \langle \mathcal{S}, \mathcal{A}, \mathcal{O}, \mathcal{T} \rangle, where \mathcal{S} is the persistent state space of all underlying storage engines (databases, filesystems, external services), \mathcal{A} is the space of tool invocations, \mathcal{O} is the space of tool return payloads (observations) returned to the agent, and \mathcal{T}: \mathcal{S} \times \mathcal{A} \to \mathcal{S} \times \mathcal{O} is the environment transition function. A workflow execution over T steps induces a trajectory \tau = \langle (s_0, a_1, o_1, s_1), \dots, (s_{T-1}, a_T, o_T, s_T) \rangle. Across this trajectory, the four concepts in your prompt correspond to strictly nested formal guarantees.

At **Level 1 (Tool Invocation Success — Syntactic & Transport Layer)**, at step t, the agent emits an action a_t \in \mathcal{A} that satisfies the syntactic schema of the tool interface (valid JSON arguments, types, and permissions), and the tool adapter returns an observation o_t \in \mathcal{O} without raising a transport or runtime exception (such as HTTP 200 OK or {"status": "success"}). Crucially, Level 1 only establishes that a *message was exchanged*. Recent benchmark contract audits demonstrate that tools frequently exhibit **Ungrounded Oracles** (returning a success string while executing a documented no-op, as uncovered in MedAgentBench) or **Silent Tool Errors** (returning plausible, schema-valid, but factually corrupted data, as formalized in RITE).

At **Level 2 (Operation Success — Local Hoare Triple Satisfaction)**, the concrete state transition s_{t-1} \xrightarrow{a_t} s_t satisfies the local Hoare triple \{P_t(s_{t-1})\}\ a_t\ \{Q_t(s_{t-1}, a_t, s_t)\}. That is, the persistent backend actually materialized the intended atomic write in s_t. However, Level 2 is purely local in both time and scope: it does not prevent a local operation from firing an unapproved database trigger (as demonstrated in EffectMatch),

nor does it prevent step t+3 from overwriting or invalidating the state achieved at step t.
At **Level 3 (Workflow Success — Global Postcondition, Trace Provenance, & Frame Invariant)**, across the entire trajectory \tau from s_0 to s_T, the execution satisfies a formal workflow contract \mathcal{C} = \langle \mathcal{P}, \mathcal{R}, \mathcal{T}_{\text{spec}} \rangle. First, under *Target Postconditions (\mathcal{P})*, the terminal state s_T and canonical state difference \Delta S = \text{Diff}(S_0, S_T) satisfy all required entity creations, updates, deletions, and cross-store consistency predicates, including exact record counts. Second, under the *Frame Invariant / Zero Unaccounted Residue (\mathcal{R})*, every mutation in \Delta S is accounted for by an authorized clause in \mathcal{P} (\Delta S \setminus \Delta S_{\text{claimed}} = \emptyset), and no aborted speculative branch leaves uncompensated external effects. Third, under *Causal & Procedural Provenance (\mathcal{T}_{\text{spec}})*, the trace \tau satisfies required temporal ordering constraints (a_i \prec a_j) and witness grounding, meaning every value written in s_T or reported to the user is causally derived from valid prior observations o_{1..t}.
Finally, at **Level 4 (User-Intent Satisfaction — Pragmatic & Semantic Alignment)**, the formal contract \mathcal{C} itself is semantically equivalent to the human user's true, often underspecified natural-language intent u \in \mathcal{U}. In formal verification of LLM agents, translating natural language u into a formal specification \mathcal{C} = \tau_{\text{NL}\to\mathcal{F}}(u) is known as the **Translation Bottleneck**. Because automated translators and human contract authors frequently produce syntactically valid contracts that miss implicit user requirements (\tau_{\text{NL}\to\mathcal{F}}(u) \not\equiv u), an agent can achieve 100% Level 3 Workflow Success while violating Level 4 User Intent—a failure mode termed the **False-Assurance Problem**.



A.2 What Information Is Fundamentally Necessary to Verify Task Completion?

From the standpoint of formal runtime verification and transaction processing, an independent

arbiter cannot establish Level 3 Workflow Success without four irreducible pieces of evidence:

1. **The Pre-Execution Baseline Snapshot (S_0)**: The complete state of all mutable entities within the environment's scope immediately prior to the agent's first action. Without S_0, a verifier seeing a valid record in S_T cannot determine whether the agent created it, whether it already existed before the run (a no-op pass on a pre-populated workspace), or which unrelated records were mutated relative to their starting values.
2. **The Post-Execution Terminal Snapshot (S_T)**: The complete state of all mutable entities after the agent signals termination (or exhausts its step budget), captured via an out-of-band, read-only administrative channel that bypasses the agent's own tool wrappers.
3. **The Typed Causal Execution Trace (\tau)**: An append-only sequence of timestamped (or vector-clocked) events \tau = \langle e_1, \dots, e_n \rangle capturing user inputs, tool call invocations (with bound arguments), raw tool return payloads, and transaction boundaries.
4. **An Executable Task & Boundary Contract (\mathcal{C}, \Omega)**: A formal specification defining (a) required positive state assertions and cardinalities, (b) the allowed mutation frame, (c) temporal and data-flow trace constraints, and (d) the **Observation Boundary Map (\Omega)** declaring which state surfaces and external channels are fully observable versus partially observable.

A.3 & A.4 Can Final-State Inspection Alone Establish Success? What Happens When Two Execution Histories Produce the Same Final State?

No. Final-state inspection (\Delta S = \text{Diff}(S_0, S_T)) is necessary for verifying *extensional* properties (what persists in storage), but it is mathematically blind to *intensional* trace properties and *hyperproperties* (how the state was computed and what information flowed during execution). When two distinct execution histories \tau_1 \neq \tau_2 start from S_0 and produce the exact same terminal state S_T^{(1)} = S_T^{(2)}, final-state inspection must assign them the identical verdict. In practice, one trajectory may be genuinely correct while the other represents a severe failure across four mechanisms documented in recent literature.

First, final-state checks miss **Epistemic Shortcuts and Unwitnessed Hallucinations**. Suppose a contract requires the agent to read a student's raw exam scores from a CSV file, compute the weighted average (82.5), update the SQLite marks column to 82.5, and write a summary report. In \tau_1, the agent reads the CSV, computes 82.5, and updates the database. In \tau_2, the agent never calls read_file (or ignores a corrupted tool output), guesses 82.5 from a prior prompt example or parametric memory, and writes 82.5 to the database. Both runs yield the exact same S_T. As demonstrated in AgentLTL (2026) and APIFlow-Bench (2026), grading S_T alone rewards lucky guesses and parametric recall, which fail silently the moment underlying data shifts.

Second, final-state checks cannot see **Negative-Space Procedural Omissions**. In Vigil (2026), researchers documented an insidious failure mode in production skill pipelines: an upstream tool generates an intermediate motion-transform artifact M, and a global composition policy requires calling a validation tool on M before a downstream skill consumes it to generate a mask. If the agent skips the intermediate validation call, the final generated mask still satisfies every JSON schema check in S_T. The violation leaves no trace in any single command or final schema check; it is woven into the "negative space" of the execution history—a critical

validation event that never happened.
Third, terminal snapshots miss **Dirty Intermediate States and Premature Externalization (Anomalies A5 & A7)**. Suppose in \tau_2 the agent modifies an unrelated student's record (student_id = 99) at step 2, realizes its mistake at step 4, and updates student_id = 99 back to its original value from S_0. At workflow end, \text{Diff}(S_0, S_T) shows zero net change to student_id = 99. In an isolated single-threaded sandbox, this recovery may be acceptable; however, in a realistic environment with database triggers, audit logs, or concurrent readers, that transient dirty write may have already fired an external notification or been read by another worker—a failure mode formalized by Trofimov and Novikov (2026) as **Premature Externalization (A5)** and **Conflicting Externalization (A7)**.
Fourth, storage diffs are completely blind to **Read-Only Data Exfiltration and Privacy Breaches**. If a prompt-injected agent reads the entire students table and transmits sensitive personal data via an HTTP GET query parameter (evil.site/exfil?data=...) or an unauthorized read-only lookup tool, the local SQLite database and report file in S_T are 100% identical to a benign run. Only trace inspection (\tau) can detect read-only confidentiality, consent, and purpose-limitation violations.

A.5 How Can We Detect Unwanted Side Effects When They Are Not Explicitly Listed?

Attempting to enumerate every possible unwanted side effect in a task contract (do not modify student 2, do not delete table grades, do not create temp.txt) is a classic anti-pattern known in formal AI as the **Frame Problem**. You solve this by inverting the verification logic from *blacklisting forbidden side effects* to **whitelisting permitted mutations under a Closed-World Frame Invariant**.
Formally, let S_0 and S_T be complete snapshots of all tables, rows, and columns in the SQLite database and all files in the workspace directory. We compute the canonical state difference as a set of atomic mutation tuples:
\Delta S = \text{Diff}(S_0, S_T) = \Delta_{\text{insert}} \cup \Delta_{\text{update}} \cup \Delta_{\text{delete}}
$where each element $d \in \Delta S$ identifies the storage surface, entity or table, primary key, field name, old value, and
\Delta S_{\text{residue}} = \Delta S \setminus \bigcup_{p_k \in \mathcal{P}} \Delta S_{\text{claimed}}(p_k)
Under the **Closed-World Invariant** (implemented in AppWorld and formalized in Agent-Diff), the frame condition requires \Delta S_{\text{residue}} = \emptyset (modulo an explicit environment-level ignore list for benign system metadata like auto-increment sequence counters or access timestamps). If the agent modifies an unrelated student record, inserts a duplicate row, or leaves an unrequested scratch file on disk, those mutations remain in \Delta S_{\text{residue}}, triggering an automatic **Residue Failure** without the contract author ever having to anticipate which unrelated record the agent might touch.

A.6 Precise Distinction Between Incomplete, Incorrect, and Unverifiable Tasks

Using our formal sets—positive postconditions \mathcal{P}, frame/residue invariant \mathcal{R} (\Delta S_{\text{residue}} = \emptyset), trace provenance constraints \mathcal{T}_{\text{spec}}, and the observation boundary map \Omega—we can define these three failure states without

ambiguity:

Failure Category	Formal Condition over (\Delta S, \tau, \Omega)	Concrete Student-Workflow Example	Operational Implication
Incomplete Task *(Liveness Deficit)*	\forall c \in \mathc[span_58](start_span)[span_58](end_span)[span_64](start_span)[span_64](end_span)al{P} \cup \mathcal{R} \cup \mathcal{T}_{\text{spec}}, c is observable under \Omega; \Delta S_{\text{residue}} = \emptyset (no unauthorized side effects); at least one p \in \mathcal{P} is satisfied, but \exists p' \in \mathcal{P} such that p' is **unmet** (missing expected state change).	Agent creates the student record and updates marks to 85 in SQLite, but stops before generating and saving the Markdown report file on disk.	Safe to resume or prompt the agent to complete the remaining sub-goal from the current state S_T.
Incorrect Task *(Safety / Value / Frame Violation)*	Observable evidence demonstrates either: (1) a target entity was created/updated with wrong field values or wrong multiplicity (count != expected), (2) \Delta S_{\text{residue}} \neq \emptyset (collateral damage), or (3) \tau \not\models \mathcal{T}_{\text{spec}} (ungrounded value or violated ordering).	Agent updates the student's marks to 95 instead of 85, inserts two duplicate student records, or modifies an unrelated student's row.	Cannot simply continue; requires state rollback or compensation to undo corrupted state before retry.
Unverifiable Task *(Epistemic / Boundary Deficit)*	No observable check evaluates to False (0) and observable \Delta S_{\text{residue}} = \emptyset, **but** \exists c \in \mathcal{P} \cup \mathcal{R} \cup \mathcal{T}_{\text{spec	Agent creates the student record, updates marks, saves the report, and calls send_parent_notification_webhook(), which times out (HTTP 504) with no status query	Runtime must **abstain from committing** downstream work and escalate to human review or an out-of-band reconciliation protocol.

Failure Category	Formal Condition over (\Delta S, \tau, \Omega)	Concrete Student-Workflow Example	Operational Implication
	}} whose evaluation under \Omega yields \text{Unknown} (\frac{1}{2}) due to missing readback, timeout, or uninstrumented external state.	API.	

A.7 Under What Conditions Is Independent Verification Impossible?

Theoretical analysis by Trofimov and Novikov (Sep 2026) and benchmark contract audits demonstrate that independent post-execution verification is **fundamentally impossible** under four conditions:

1. **Unacknowledged Fire-and-Forget External Boundaries (Anomalies A1, A2, A3)**: When a tool mutates state in an external third-party service that provides neither a durable acknowledgment nor an idempotent status-lookup endpoint, a network timeout leaves the true world state permanently in doubt. An external verifier cannot determine whether the operation occurred—risking **Missing Committed Effect (A2)** if optimistically assumed successful, **Duplicated Effect (A1)** if retried without idempotency, or **Orphaned Compensation (A3)** if pessimistically refunded when the original payment never went through.
2. **Open-World Exogenous Reactions (Anomaly A8 — Phantom Compensation)**: Once an agent externalizes an effect (such as publishing a grade or sending an offer email) and an external human or third-party automated system observes it and takes an independent action, rolling back or compensating the local database state cannot retract the external reaction.
3. **Absence of a Deterministic Oracle or Reference State**: If a task's success criterion is purely subjective ("write an empathetic email") or depends on ephemeral live external data that was not captured in the trace \tau at read time, no deterministic state checker can establish correctness without falling back to a heuristic LLM judge.
4. **Violation of Complete Mediation or Verifier Isolation**: If the agent can mutate the environment through an unmonitored side channel (such as raw shell execution or unregistered database triggers outside the verifier's snapshot scope) or can read and tamper with the verifier's own test harness (as observed in 2026 frontier coding rollouts where an agent ran ls -la /logs/verifier /logs/agent to inspect grader artifacts), independent verification fails.

A.8 Should AURA Inspect Intermediate States, Final States, Execution Traces, or a Combination?

AURA should inspect a combination of Initial and Final State Snapshots (S_0, S_T) and the Typed Execution Trace (\tau), augmented by lightweight transaction-local audit triggers for intermediate mutations. Final states (S_T) alone cannot detect whether S_T changed relative to S_0, cannot catch epistemic shortcuts (hallucinated right values), and

cannot detect dirty intermediate writes that were subsequently reverted. Initial-plus-final state diffs ($\Delta S = \text{Diff}(S_0, S_T)$) catch all persistent positive outcomes and persistent collateral damage, but remain blind to data-flow provenance (did the agent actually read the source record?), negative-space procedural omissions (did the agent skip a mandatory validation check?), and read-only exfiltration. Conversely, taking full filesystem and database copies after every single tool call incurs unnecessary I/O overhead when simpler mechanisms suffice. In Python and SQLite, attaching lightweight **SQLite AFTER INSERT/UPDATE/DELETE audit triggers** records every intermediate row mutation into an append-only verifier audit table during the run at sub-millisecond cost. Combining $\Delta S = \text{Diff}(S_0, S_T)$ with the typed tool trace τ and an append-only SQLite mutation log gives you complete observability over both final outcomes and intermediate transient mutations.

2. Independent Verdict: Is AURA Worth Building, and Why?

Evaluating your proposal critically and without flattery, **the underlying problem is genuinely important, and a refined version of AURA is well worth building—but only if you fix its conceptual redundancies and experimental confounds first.**

Regarding **problem scope and technical feasibility**, framing AURA broadly as "an engine that verifies whether an AI agent genuinely completed a user's multi-step task" is too broad if taken to include open-ended text generation or live web browsing without deterministic oracles. However, because you explicitly scope the initial system to **machine-checkable JSON task contracts over structured storage**, the scope is tight and tractable. Technically, a clean Python + SQLite + filesystem state-diff and trace verifier is 100% feasible and can be built in **2–3 days on a normal laptop** using only the Python standard library (sqlite3, hashlib, difflib, json, pathlib) and pydantic. The primary architectural complication to strip out is your division of verification into five separate dimensions (Completion, Correctness, Consistency, Residual effects, Uncertainty), which conflates positive state predicates with negative frame invariants and epistemic truth values.

Why can't existing **ACID database transactions, assertions, test frameworks, or workflow engines** solve this problem on their own?

- *ACID Transactions*: SQLite transactions guarantee *atomicity* (all-or-nothing execution) and enforce schema-level invariants (FOREIGN KEY, UNIQUE, CHECK). However, a database engine has no awareness of **task-level user intent**: it cannot know that an agent updated the wrong student's marks, forgot to generate a report file on the filesystem, or committed a local transaction after an external REST API silently failed.
- *Workflow Engines and Saga Orchestrators*: Traditional workflow engines (such as Temporal or Airflow) and classical Sagas assume **trusted, deterministic worker code** where an HTTP 200 OK means the business logic succeeded. When an LLM agent is the worker, the worker itself may hallucinate arguments, skip steps, or misinterpret a silent tool error.
- *Test Frameworks (pytest) vs. Runtime Arbiters*: We must be candid—at its mechanical core, a post-execution verifier evaluating a JSON task contract over database snapshots is **a declarative integration test runner paired with a sandbox state differ**. The reason a dedicated engine (an independent runtime arbiter) is required in agentic systems is that verification is not merely a build-time CI check; it is an **online runtime commit gate** that must decide in real time whether to commit the agent's workspace, roll it back, or escalate

to a human.

Are **PASS, FAIL, and UNCERTAIN sufficient**? They are necessary and sufficient as **top-level gating decisions** (mapping to *Commit*, *Abort/Rollback*, and *Hold/Escalate*). However, a single flat FAIL is insufficient for automated recovery: an orchestrator needs structured diagnostic sub-codes to know whether the agent merely stopped early (FAIL_INCOMPLETE, where resuming from S_T is safe) or corrupted an unrelated record (FAIL_RESIDUE, where rolling back to S_0 is mandatory).

Finally, **what assumptions must be made about AURA's access, permissions, and trustworthiness, and could AURA itself make incorrect decisions?** AURA requires **privileged out-of-band read access** to the raw storage engines, **complete snapshot coverage** of the declared observation boundary, and **strict process isolation** so the agent cannot modify AURA's snapshots or contracts. Even with privileged access, **AURA can easily make incorrect decisions if its contracts are flawed.** Recent 2026 audits of shipped deterministic benchmark verifiers demonstrate that human-authored contracts routinely suffer from **false accepts** (for instance, forgetting to check record counts when an agent duplicates a non-idempotent write) and **false rejects** (rejecting valid alternative execution paths, equivalent string formatting such as \n vs. literal newlines, or valid boundary dates). Auditing AURA's own contract adequacy via mutation testing is therefore essential.

Evaluation Axis	Strongest Arguments FOR Building AURA	Strongest Arguments AGAINST Building AURA (As Currently Proposed)
Scientific & Technical Merit	Directly targets the failure modes dominating 2026 agent research: silent tool corruption (RITE), unfaithful status reports (CTXGAP), and unapproved persistent side effects (EffectMatch).	Pure state-diff verification against JSON contracts (Agent-Diff) and SQLite collateral-damage testing (AppWorld) **already exist in published 2024–2026 literature**. Building a clone without a new angle is not novel.
Architectural Necessity	LLM self-verification is unreliable due to shared generative blind spots and context degradation. An external, deterministic arbiter is required to establish a trusted commit boundary.	In a local SQLite + filesystem sandbox, observability is 100% white-box. Unless you explicitly model external boundary failures or revoked readback permissions, the UNCERTAIN state is artificial and will never trigger naturally.
Experimental Validity	Controlled failure injection in a deterministic SQLite and filesystem simulator allows exact measurement of false-accept and false-reject rates without noisy LLM-judge variance.	Your proposed baseline comparison (Approach A: Trust Agent vs. Approach B: Tool Check vs. Approach C: AURA) is **confounded as stated** because it gives AURA privileged database read access denied to the baselines.
Student Career & Practical	Teaches high-demand	Spending weeks building an

Evaluation Axis	Strongest Arguments FOR Building AURA	Strongest Arguments AGAINST Building AURA (As Currently Proposed)
Value	production engineering skills: transactional state diffing, design-by-contract, runtime monitoring, and adversarial fault injection—all with zero GPU or paid API budget.	elaborate "framework" before running an information-matched ablation study risks creating over-engineered boilerplate rather than a focused empirical contribution.

3. Existing Research: What Has Already Been Done, with Sources?

Before writing code, you must recognize that **outcome-centric, state-diff-based verification with collateral-damage detection is already an established paradigm in the AI agent literature**. Below is a synthesis of the state of the art through October 2026, separating verified facts in published systems from open gaps.

Detailed Five-Point Analysis of Closely Related Works

1. AppWorld: A Controllable World of Apps and People for Benchmarking Interactive Coding Agents (2024)

- **Citation & Links**: Harsh Trivedi et al. (ACL 2024 Best Resource Paper). Paper: https://arxiv.org/abs/2407.18901 | Code: https://github.com/StonyBrookNLP/appworld.
- **1. What problem it solves**: Evaluates interactive coding and API-calling agents across 9 simulated everyday applications (Gmail, Venmo, Amazon, local filesystem, etc., backed by 457 APIs and 101 SQLite tables populated with ~370,000 rows across 750 tasks) where tasks require multi-step state manipulation rather than static text matching.
- **2. How its approach works**: Uses serverless in-memory SQLite and FastAPI. Before an agent runs, AppWorld snapshots the initial database state (S_0). After the agent calls complete_task(), an evaluation suite of programmatic, state-based unit tests (averaging 8 tests, up to 22 per task) uses hash-based database diffing to localize changes at the table, row, and column levels, checking both that expected state transitions occurred and that no unexpected changes (**collateral damage**) disturbed unrelated tables or rows.
- **3. Whether it already solves your proposed problem**: **Largely yes, for benchmark evaluation.** AppWorld already implements Python + SQLite state-diff verification and explicitly rejects runs that cause collateral damage (your "Residual effects" dimension).
- **4. What limitations remain**: First, AppWorld's verifiers are hand-written imperative Python test suites tied to its 250 benchmark scenarios rather than a general declarative arbiter. Second, an October 2026 mutation audit by Richard Abrich (https://arxiv.org/abs/2610.09142) discovered **record-count blindness** in shipped AppWorld verifiers: when an agent duplicated a non-idempotent write (creating an extra payment request or player record while preserving checked field values), AppWorld falsely returned PASS in **6 of 15 constructed fault effects** (40.0%, spanning all 3 daily variants in 2 of 5 eligible scenario generators). Third, AppWorld is purely state-based; it does not verify trace provenance or handle partial observability (UNCERTAIN).

- **5. What AURA could contribute beyond it**: Enforcing mandatory cardinality and multiplicity invariants in declarative contracts to eliminate duplicate-write blind spots, coupling state diffs with causal trace provenance, and introducing a formal UNCERTAIN epistemic state for unconfirmed external boundaries.

2. Agent-Diff: Benchmarking LLM Agents on Enterprise API Tasks via Code Execution with State-Diff-Based Evaluation (2026)

- **Citation & Links**: Hubert M. Pysklo, Artem Zhuravel, and Patrick D. Watson (arXiv 2026). Paper: https://arxiv.org/abs/2602.11224 | Code: https://github.com/agent-diff-bench/agent-diff | Platform: https://www.agentdiff.dev/.
- **1. What problem it solves**: Evaluates code-executing LLM agents across containerized replicas of enterprise APIs (Slack, Linear, Box, Google Calendar; 108 endpoints, 224 tasks) without relying on brittle action-trace matching or stochastic LLM judges.
- **2. How its approach works**: Snapshots database entity tables before (S_{t_{\text{start}}}) and after (S_{t_{\text{end}}}) execution to compute a canonical JSON state diff \Delta S = \{\text{inserted, deleted, updated entities}\}. Task contracts are written as **declarative JSON assertions** specifying diff_type (added, deleted, updated), entity, where field constraints, and expected_count. Crucially, Agent-Diff enforces a strict **Closed-World Invariant**: every mutation in \Delta S must be accounted for by a contract assertion, or the run automatically fails.
- **3. Whether it already solves your proposed problem**: **Yes, for the state-diff + JSON contract prototype you described in Section 3.** Agent-Diff's declarative JSON state-diff contract and closed-world invariant already implement AURA's proposed Completion, Correctness, Consistency, and Residual effects checks.
- **4. What limitations remain**: Agent-Diff intentionally treats the execution process as a black box—evaluating *only* \Delta S. Consequently, it cannot detect **epistemic shortcuts** (an agent guessing a value without reading the required source file), **negative-space procedural violations** (skipping a required intermediate validation call), **transient dirty writes** that were overwritten before t_{\text{end}}, or **partial observability** where UNCERTAIN is required.
- **5. What AURA could contribute beyond it**: Moving from a *State-Diff-Only* contract to a **Dual-Surface (State-Diff + Trace-Provenance) + Epistemic Boundary** contract, and demonstrating empirically via ablation where state-diff-only verification fails.

3. EffectMatch — Beyond Approved Actions: Runtime Validation of Persistent Outcomes in Agent Workflows (2026)

- **Citation & Links**: Haoran Zhang et al. (Submitted to IEEE Transactions on Software Engineering, Sep 2026). Paper: https://arxiv.org/abs/2609.31301.
- **1. What problem it solves**: Closes the "approval-to-persistent-outcome gap," where a human or policy approves an agent's proposed tool call (such as updating an invoice status from pending to paid), the tool call succeeds, but a hidden database trigger, cascading update, or server-side rule writes an unapproved persistent side effect (such as inserting an unapproved notification row) or a lost network reply causes a duplicate payment on retry.
- **2. How its approach works**: Introduces the **Effect Commit Contract** and an **Effect Manifest** specifying target objects, permitted changes, and relations that must remain

unchanged within a declared **observation boundary**. Before executing a write, EffectMatch checks live backend metadata to certify that all reachable persistent-effect routes (including triggers) are covered; it executes the command inside a held-open private transaction, inspects transaction-local audit records to verify \text{EffectExact}_p(A, E) (comparing all fields and multiplicities), and commits or aborts before releasing downstream workflow steps. Across 206 public business tasks, it preserved all clean executions and prevented all tested incorrect commits.

- **3. Whether it already solves your proposed problem**: **Partially.** It solves online, pre-commit persistent-outcome validation for individual database operations and formalizes the observation boundary.
- **4. What limitations remain**: Holding private database transactions open across an entire multi-step agent workflow is often infeasible when steps interleave local database writes, local file generation, and external REST API calls. Furthermore, EffectMatch validates individual approved operations rather than end-to-end multi-step workflow contracts with data-flow provenance across heterogeneous stores.
- **5. What AURA could contribute beyond it**: Extending EffectMatch's **Observation Boundary** and **Effect Manifest** concepts to end-to-end, heterogeneous multi-step workflows (SQLite + filesystem + external webhooks) with an explicit UNCERTAIN epistemic calculus when routes cannot be held in a single open transaction.

4. When Tool Calls Succeed but Workflows Fail: Anomalies at the Agent–Tool Boundary (2026)

- **Citation & Links**: Artem Trofimov and Boris Novikov (Sep 2026). Paper: https://arxiv.org/abs/2609.15397.
- **1. What problem it solves**: Formalizes why locally successful tool calls fail to compose into globally consistent workflows under retries, speculative execution, concurrency, and partial failures, and audits 98,291 tools across registered Model Context Protocol (MCP) servers to measure whether current tool schemas expose the metadata needed for verification and recovery.
- **2. How its approach works**: Separates external-world events (extern[span_329](start_span)[span_329](end_span)alize(a, e)) from runtime observations (observe(a, outcome)) and defines **eight recurring external-effect anomalies**:
  - *Uncertainty Family (A1–A3)*: **A1 Duplicated Effect** (unconfirmed call re-issued, executing twice), **A2 Missing Committed Effect** (unconfirmed call optimistically assumed successful, workflow commits without required effect), **A3 Orphaned Compensation** (unconfirmed call pessimistically compensated, issuing a spurious refund).
  - *Workflow Lifecycle Family (A4–A6)*: **A4 Uncompensated Residue** (aborted workflow leaves surviving effect because compensation failed or did not exist), **A5 Premature Externalization** (effect observed externally before workflow resolves and later aborts), **A6 Contaminated Speculation** (committed effect depends on a speculative branch that later aborts).
  - *Interaction Family (A7–A8)*: **A7 Conflicting Externalization** (concurrent unmediated non-commuting writes), **A8 Phantom Compensation** (effect compensated locally, but an external actor already reacted to it).
- **3. Whether it already solves your proposed problem**: **No—it is a theoretical and**

empirical census paper, not a verification engine. However, it provides the exact formal vocabulary (Uncertainty A1–A3 and Residue A4–A6) that your AURA proposal currently describes only intuitively.

- **4. What limitations remain**: The authors show that standard MCP tool annotations only provide coarse call-level hints and none of the evaluated runtimes (Atomix, Cordon, ACRFence, RAC, CoAgent, Shepherd) fully covers A1–A8.
- **5. What AURA could contribute beyond it**: You can ground AURA directly in Trofimov and Novikov's anomaly catalog—designing AURA specifically as an arbiter that detects and distinguishes **A1 (Duplicated Effect)**, **A2 (Missing Committed Effect / Uncertainty)**, **A3 (Orphaned Compensation)**, and **A4 (Uncompensated Residue)**.

5. Vigil (2026) and AgentLTL (2026): Formal Trace & Provenance Verification

- **Citations & Links**:
  - **Vigil**: *Behavioral Enforcement by SMT Solving* (Jun 2026). Paper: https://arxiv.org/abs/2606.26524.
  - **AgentLTL**: Laïla Elkoussy and Julien Perez, *AgentLTL: A Trace-Verification Framework for Measuring, Enforcing, and Training Procedural Compliance in Tool-Using LLM Agents* (Jul 2026). Paper: https://arxiv.org/abs/2607.02599.
- **1. What problem they solve**: Catch procedural, temporal, and grounding violations where an agent produces a valid-looking final answer or state S_T via an invalid process (such as skipping intermediate artifact validation in Vigil, or answering from parametric memory without reading required files in AgentLTL).
- **2. How their approaches work**: Vigil compiles behavioral policies with finite-trace temporal semantics into SMT queries over typed event traces, recovering the exact offending tool invocation from the solver's unsatisfiable core (reaching 95.8% recall at 89.6% precision across 152 labeled runs and 216 real skill-bundle executions). AgentLTL evaluates traces against First-Order Linear Temporal Logic (FO-LTL) formulas covering ordering (\kappa_1), call presence (\kappa_2), and **trace grounding** (\kappa_3: every entity cited in the final output must have a witness in a prior tool result).
- **3. Whether they already solve your proposed problem**: **No.** Both operate strictly over the execution trace \tau. As EffectMatch and Do Agent Benchmarks Do What They Say? demonstrate, trace-only monitors trust the tool's return payload and cannot see silent database corruption, unapproved database triggers, or partial persistent writes.
- **4. What limitations remain**: Lack of direct persistent storage snapshot diffing (\Delta S) and closed-world residue checking.
- **5. What AURA could contribute beyond them**: Combining AgentLTL and Vigil-style **trace witness grounding** with Agent-Diff and [span_175](start_span)[span_175](end_span)EffectMatch-style **persistent state diffing**.

6. RITE (2026) and CTXGAP (2026): Silent Tool Errors and Unfaithful Agent Reports

- **Citations & Links**:
  - **RITE**: Wenshuo Zhang et al., *RITE: Evaluating End-to-End Resilience to Silent Tool Errors in E-Commerce Agentic Systems* (Oct 2026). Paper: https://arxiv.org/abs/2610.27777.
  - **CTXGAP**: Guoyang Xu et al., *What Coding Agents Leave Unsaid: Measuring*

Task-State Gaps Between Execution and Reports (Oct 2026). Paper: https://arxiv.org/abs/2610.20003.

- **1. What problem they solve**: Quantify why your "Approach A (Trust tool responses and agent self-reports)" fails in real deployments.
- **2. How their approaches work**: RITE constructs 167 matched workflow units (501 instances across correct, implicit/silent-error, and explicit-error conditions) from production e-commerce logs, injecting plausible but incorrect tool outputs without error codes. Across 10 LLMs, silent tool errors roughly halve end-to-end task success, prove consistently harder to handle than explicit exceptions, and are noticed by models in only **20.8% to 33.6% of runs** (meaning 66.4% to 79.2% slip by unnoticed, as detailed in the study and its technical review). CTXGAP probes 37 coding tasks (4,799 valid probes) and finds that while full-context readers recover 92.7% of task-state answers, report-only readers recover only 52.0%—a **40.7 percentage point gap** (leaving 48.0% of task-state questions unrecovered from the agent's report alone).
- **3. Role for AURA**: These two October 2026 papers provide empirical motivation for your introduction and directly inform your failure-injection taxonomy.

Summary Matrix of Broader Related Work Across All Eight Requested Areas

Research Category	Representative Works, Years, & Verifiable Links	Core Mechanism	What Remains Unsolved for AURA
1. Stateful Agent Benchmarks	• **WorkBench** (2024; Revisited 2026): https://arxiv.org/abs/2405.00823 & https://arxiv.org/abs/2606.13715 • **\tau-bench / \tau^2-bench** (2024/2025): https://arxiv.org/abs/2406.12045 • **SWE-bench** (2024): https://arxiv.org/abs/2310.06770 • **WebArena** (2024): https://arxiv.org/abs/2307.13854 • **OSWorld** (2024): https://arxiv.org/abs/2404.07972 • **WorkArena** (2024): https://arxiv.org/abs/2403.07718 • **AgentBench** (2024): https://arxiv.org/abs/23	Evaluate agents via final database state hashes (\tau-bench), exact sandbox table matching (WorkBench), or task-specific Python/Playwright evaluation scripts (OSWorld, WebArena, WorkArena).	Evaluators are tightly coupled to static benchmark tasks; binary PASS/FAIL only; no formal epistemic UNCERTAIN state; susceptible to false accepts on duplicate writes and false rejects on valid alternative paths.

Research Category	Representative Works, Years, & Verifiable Links	Core Mechanism	What Remains Unsolved for AURA
	08.03688		
2. Interactive & Graph-Based Task Evaluators	• **ToolSandbox** (2024/2025): https://arxiv.org/abs/2408.04682 • **Interlock** (Oct 2026): https://arxiv.org/abs/2610.19686 • **APIFlow-Bench** (2026): https://blog.postman.com/apiflow-bench/	ToolSandbox scores trajectories over a DAG of **Milestones** (required state/trace events) and **Minefields** (forbidden actions). Interlock models tasks as typed completion graphs with grounding, coupling, and realization edges. APIFlow-Bench traces "earned canary values" across state and answer surfaces.	Do not address partial observability boundaries (UNCERTAIN) or transaction-level unapproved side-effect triggers (EffectMatch).
3. Formal Methods & Runtime Enforcement DSLs	• **AgentSpec** (ICSE 2026): https://arxiv.org/abs/2503.18666 • **Agent Behavioral Contracts (ABC)** (Feb 2026): https://arxiv.org/abs/2602.22302 • **VeriGuard** (Oct 2025): https://arxiv.org/abs/2510.05156 • **Lean4Agent** (NeurIPS 2026): https://arxiv.org/abs/2606.06523 • **CPL / ZipperGen** (Sep 2026): https://arxiv.org/abs/2605.20923	Specify preconditions, hard/soft invariants, and temporal guards via DSLs (AgentSpec, ABC) or theorem provers (Lean4Agent, VeriGuard), intercepting actions at runtime with millisecond overhead.	Focus primarily on **action-level pre-execution gating** or behavioral drift bounds rather than independent post-execution storage-diff arbitration. Also face the **Verifier Tax** (blocking unsafe actions without helping the agent complete the task).
4. Transactional Runtimes & Rollback Systems	• **SagaLLM** (VLDB 2025): https://arxiv.org/abs/2503.11951 • **Atomix** (2026): https://arxiv.org/abs/2602.14849 • **GoEX** (2024): https://arxiv.org/abs/2404.06921 • **AgentRewind** (Aug	Provide compensating transactions (SagaLLM, Atomix), post-facto undo/damage confinement (GoEX), or aligned context + filesystem/process checkpointing (AgentRewind, Planarian via CRIU + ZFS).	Provide the **recovery mechanism** (rollback, statepoints, compensation), but rely on an external oracle or human user to decide *when* a completed workflow state is semantically invalid or contaminated by residue.

Research Category	Representative Works, Years, & Verifiable Links	Core Mechanism	What Remains Unsolved for AURA
	2026): https://arxiv.org/abs/2608.14380 • **Planarian** (Sep 2026): https://arxiv.org/abs/2609.35366		
5. Judge-Based & Guardrail Frameworks	• **Agent-as-a-Judge** (ICML 2025): https://arxiv.org/abs/2410.10934 • **LlamaFirewall** (May 2025): https://arxiv.org/abs/2505.03574 • **NeMo Guardrails** (2023): https://arxiv.org/abs/2310.10501 • **GuardAgent** (ICML 2025): https://arxiv.org/abs/2406.09187	Use tool-augmented LLM judges (Agent-as-a-Judge) or prompt, chain-of-thought, and static code scanners (LlamaFirewall) to evaluate trajectories.	Stochastic, vulnerable to judge bias and kappa deflation, and vulnerable to reasoning-extension Denial-of-Service attacks (arXiv:2606.14517).
6. Verifier & Benchmark Validity Audits	• **Finding Blind Spots in AppWorld & WorkArena** (Oct 2026): https://arxiv.org/abs/2610.09142 • **Benchmarking the Benchmarks** (Jun 2026): https://arxiv.org/abs/2607.02577 • **Executable-Contract Audit** (Sep 2026): https://arxiv.org/abs/2609.37315	Audit shipped verifiers using mutation testing (Abrich 2026), human trace adjudication (Vaghasiya et al. 2026), and pre/post tool contract checks (Bellibatlu et al. 2026).	Expose widespread flaws in current verifiers (18.5% human-verifier disagreement across 496 tasks; duplicate-write blindness; ungrounded oracles), creating a direct research opening for **mutation-hardened verification contracts**.

4. Weaknesses: What Is Wrong or Incomplete in Your Current Thinking?

To turn AURA into a defensible research project, you must address six foundational weaknesses in your current proposal:

1. The Five Verification Dimensions Are Orthogonal in Name Only

You currently propose five verification dimensions: Completion, Correctness, Consistency, Residual effects, and Uncertainty. From a formal methods perspective, **these five categories are not mutually exclusive dimensions; they conflate polarity (positive vs. negative requirements) with epistemic truth values**. Specifically, Completion ("Did the student record and report get created?"), Correctness ("Are the marks 85?"), and Consistency ("Does the report text match the database row?") are all **Positive Postcondition Predicates (\mathcal{P})** evaluated over the state diff \Delta S and final artifacts S_T. Whether an assertion checks COUNT(*) == 1 (Completion), row.marks == 85 (Correctness), or report.marks == row.marks (Consistency), the underlying evaluation mechanism is identical: evaluating a first-order predicate over (S_0, S_T). Treating them as separate architectural modules will result in redundant code and arbitrary classification debates. By contrast, Residual effects is genuinely distinct: it is a **Negative Frame Invariant (\mathcal{R})** asserting that \Delta S \setminus \Delta S_{\text{claimed}} = \emptyset. Finally, Uncertainty is **not a verification dimension at all**—it is the **third truth value (\text{Unknown} / \frac{1}{2}) in a three-valued logic** that can arise when evaluating *any* positive postcondition or negative frame invariant across a degraded observation boundary.

2. The Information-Asymmetry Confound in Your Experimental Design

Your proposed three-way comparison pits **Approach A (Baseline: trust tool responses and agent self-report)** and **Approach B (Tool-Level Verification)** against **Approach C (AURA Workflow-Level Verification)**. Consider what happens when you inject a silent fault where update_marks(student_id=1, marks=85) returns {"status": "ok"} to the agent, but a bug in the tool writes marks=0 or fails to commit to SQLite. Because Approach A only inspects {"status": "ok"}, it is **structurally impossible** for Approach A to detect the failure because your experimental harness denied it read access to the database. If Approach C queries the SQLite database directly and catches the error, you have **not** demonstrated that workflow-level arbitration is a superior verification algorithm; you have merely shown that *reading the database reveals what is in the database*. To make the experiment scientifically valid, you must enforce **Information Parity**: you must compare AURA against an **Information-Matched Agentic Self-Verifier** (where the agent or an LLM judge is given the exact same readback tools read_student_record, list_all_changes, and read_file and prompted to verify its work before finishing, as tested in RITE) and against **Information-Matched Single-Surface Verifiers** (a State-Diff-Only verifier like Agent-Diff and a Trace-Only verifier like AgentLTL that have full access to their respective surfaces).

3. Blindness to Epistemic Shortcuts and Negative-Space Violations

Your proposal emphasizes checking "actual outcomes of the workflow" (records, files, outputs) rather than tool calls. While outcome checking is superior to naive tool-response trusting, **pure outcome checking is blind to how the outcome was achieved**. If the user asks the agent to "compute the student's grade from exam_raw.csv, update the database, and generate a report," and the agent fails to parse exam_raw.csv but guesses or hardcodes the right grade, or skips a mandatory compliance check, a pure state-diff verifier will output PASS. AURA must bind state changes in \Delta S to provenance witnesses in the execution trace \tau.

4. Vulnerability to Record-Count Blindness and Contract Gaming

You assume that writing JSON task contracts will cleanly separate PASS from FAIL. Yet Richard Abrich's October 2026 audit of AppWorld and WorkArena (https://arxiv.org/abs/2610.09142) showed that even expert benchmark authors write defective state contracts. Specifically, if your JSON contract checks {"table": "students", "where": {"name": "Alice"}, "assert": {"marks": 85}} using an existential quantifier (\exists r \in \text{students}), and the agent accidentally calls create_student("Alice", 85) **twice** due to a retry loop (Anomaly A1), the existential check evaluates to True and AURA outputs a **False PASS**. Similarly, in WorkArena, where an extra-field write flag was stored on the pre-submit page and lost upon form submission, 21 of 23 prospectively re-run extra-field candidates with confirmed non-default persisted values still received PASS (note that Abrich selected those 23 candidates from earlier PASS outcomes to confirm the mechanism rather than to estimate a population rate). Unless your contract schema enforces strict cardinality bounds (expected_count == 1) and full set-difference accounting (\Delta S_{\text{residue}} = \emptyset) over persisted storage, AURA will suffer from the exact same blind spots exposed in AppWorld and WorkArena.

5. The Artificiality of UNCERTAIN in a Fully Observable Local Sandbox

In a local Python + SQLite + local filesystem prototype, the verifier runs on the same machine as the database and filesystem. It has **100% deterministic, atomic read access** to every table and directory. Under what condition would a local SQLite query ever be "uncertain"? **Never**—unless you explicitly architect an **Observation Boundary Simulator** that models realistic distributed-systems observability failures:

1. Simulated external REST tools (such as an email or registrar webhook service) where network calls can return 504 Gateway Timeout (status = UNKNOWN) and no readback endpoint exists (Anomalies A1–A3).
2. Simulated permission boundaries where a legacy table or external audit log denies read access to the verifier (PermissionError on snapshot readback).
3. Ambiguous or degraded witness channels where an upstream source returns unverified metadata. Without explicitly modeling these boundary conditions, your UNCERTAIN verdict will either have 0% coverage in experiments or be misused as a fallback when a regex fails.

6. The Name "AURA"

"AURA" (*Agent Uncertainty and Residue Arbiter*) is a memorable acronym, and the words **Uncertainty** and **Residue** map directly to the two central formal concepts in the 2026 agent-boundary literature (Trofimov & Novikov's Uncertainty anomalies A1–A3 and Residue anomalies A4–A6). However, "AURA" alone sounds like a generic AI product name. Keep the acronym, but pair it with a descriptive technical subtitle in your repository and paper: **AURA: Epistemic State-Diff and Trace-Provenance Arbitration for Tool-Using Agents**.

5. Recommended Architecture: What Should AURA Actually Contain?

Designing AURA from first principles to be minimal, mathematically rigorous, and immune to the

blind spots identified in the 2024–2026 literature yields a **four-stage verification pipeline** operating over two evidence surfaces (\Delta S and \tau) and an explicit **Observation Boundary Manifest (\Omega)**.

Overview of the Four-Stage Dual-Surface Delta-Trace Pipeline

| Pipeline Stage | Core Module | Inputs | Formal Operation & Output | | :--- | :--- | :--- | :--- | | **Stage 1: Sandbox & Boundary Instrumentation** | Snapshotter & TraceProxy | Live SQLite DB, Workspace FS, Tool Adapter Calls, Boundary Map \Omega | Captures baseline snapshot S_0, terminal snapshot S_T, intermediate SQLite trigger log _aura_audit_log, and typed event trace \tau = \langle e_1, \dots, e_n \rangle tagged with obs_certainty. | | **Stage 2: Canonical Evidence Extraction** | CanonicalDiffer & ProvenanceGraph | (S_0, S_T), _aura_audit_log, \tau | Computes atomic mutation set \Delta S = \text{Diff}(S_0, S_T) = \{\Delta_{\text{insert}}, \Delta_{\text{update}}, \Delta_{\text{delete}}\} across SQLite and files; maps data-flow witnesses from tool read outputs o_t to write arguments a_{t'}. | | **Stage 3: Three-Plane Contract Evaluation** | Plane1_Postconditions, Plane2_Residue, Plane3[span_181](start_span)[span_181](end_span)[span_189](start_span)[span_189](end_span)_Provenance | \Delta S, \tau, Task Contract \mathcal{C} = \langle \mathcal{P}, \mathcal{R}, \mathcal{T}_{\text{spec}} \rangle, \Omega | Evaluates positive assertions and strict cardinalities (\mathcal{P} \to \Delta S_{\text{claimed}}), computes closed-world residue (\mathcal{R}: \Delta S_{\text{residue}} = \Delta S \setminus \bigcup \Delta S_{\text{claimed}} = \emptyset), and checks temporal ordering and witness grounding (\mathcal{T}_{\text{spec}}). | | **Stage 4: Kleene \mathcal{K}_3 Epistemic Arbitration** | KleeneArbiter & DiagnosticLedger | Truth values v(c_i) \in \{1, \frac{1}{2}, 0\} for all c_i \in \mathcal{C} | Computes global verdict V = \min_i v(c_i) \in \{\text{PASS}, \text{UNCERTAIN}, \text{FAIL}\} and emits a structured JSON Evidence Record with diagnostic recovery sub-codes (FAIL_INCOMPLETE, FAIL_RESIDUE, FAIL_UNGROUNDED, UNCERTAIN_UNCONFIRMED_EXTERNAL). |

Detailed Specification of the Four Architectural Components

1. Stage 1 & 2: Sandbox Instrumentation and Canonical Evidence Extraction

To avoid modifying agent reasoning code while maintaining complete observability, AURA wraps the execution environment at two boundaries:

- **Storage Snapshotter & SQLite Audit Triggers**: Before the agent starts (t=0), AURA takes a deterministic snapshot S_0 of all SQLite tables (dumping table schemas, primary keys, and row values/hashes) and all files in the workspace directory (recording relative paths, byte sizes, SHA-256 digests, and parsed text/JSON content). In addition, AURA attaches an out-of-band SQLite TEMP TRIGGER (AFTER INSERT, AFTER UPDATE, AFTER DELETE) on every table that logs (step_id, table_name, op_type, pk, old_json, new_json) to a protected _aura_audit_log table. This captures transient dirty writes that the agent might later revert before t=T.
- **Tool Boundary Proxy (Trace Logger)**: Every tool call emitted by the agent passes through a lightweight Python decorator or proxy that appends a typed event e_t = \langle t, \text{tool_name}, \text{args}, \text{status}, \text{return_payload}, \text{obs_certainty} \rangle to the execution trace \tau. Crucially, obs_certainty records whether the tool's outcome is CONFIRMED (such as a local SQLite or filesystem operation, or a REST API

with a verified receipt) or UNCONFIRMED (such as an external webhook that timed out or a third-party fire-and-forget endpoint without readback).

2. Stage 3: The Three Formal Verification Planes

When the agent terminates at t=T, AURA captures S_T, computes the canonical state diff \Delta S = \text{Diff}(S_0, S_T) = \{\Delta_{\text{insert}}, \Delta_{\text{update}}, \Delta_{\text{delete}}\} across both SQLite and the filesystem, and evaluates the task contract \mathcal{C} across three orthogonal planes:

1. **Plane 1 — Positive Postconditions, Cross-Store Consistency, & Cardinality (\mathcal{P})**: This plane unifies your Completion, Correctness, and Consistency dimensions into a single declarative assertion engine over \Delta S and S_T. Each assertion specifies surface (sqlite or filesystem), op (inserted, updated, deleted, exists), target (table name or file path pattern), where (key-value filter), expected_fields (exact values, numeric tolerances, or structured JSON/Markdown regexes), and a **mandatory expected_count** (defaulting to 1). *Cross-Store Consistency Assertions* bind variables across surfaces—for example, asserting that the integer parsed from report.md under regex Marks:\s*(\d+) equals students.marks where student_id == 101 in S_T. Every diff tuple d \in \Delta S that satisfies a positive assertion p_k \in \mathcal{P} (up to expected_count) is added to the claimed mutation set \Delta S_{\text{claimed}}(p_k). If actual_count > expected_count (for instance, the agent inserted two identical student records), Plane 1 flags a **Multiplicity Violation**, and the extra duplicate record remains unclaimed.
2. **Plane 2 — Closed-World Residue & Frame Invariant Engine (\mathcal{R})**: This plane computes the residual mutation set \Delta S_{\text{residu[span_182](start_span)[span_182](end_span)[span_190](start_span)[span_190](end_span)e}} = \Delta S \setminus \bigcup_{p_k \in \mathcal{P}} \Delta S_{\text{claimed}}(p_k) and evaluates two invariants. First, under *Persistent Residue*, it checks whether \Delta S_{\text{residue}} = \emptyset. If any table row was inserted, updated, or deleted outside \Delta S_{\text{claimed}}, or any extra file was left in the workspace directory, Plane 2 evaluates to False (FAIL_RESIDUE_PERSISTENT). Second, under *Transient or Uncompensated Residue*, it inspects _aura_audit_log and \tau to check whether an out-of-scope record was mutated at an intermediate step t < T or whether an external tool call was executed on an aborted branch without a corresponding compensating call (Anomaly A4/A5).
3. **Plane 3 — Causal Trace & Data-Flow Provenance Engine (\mathcal{T}_{\text{spec}})**: This plane evaluates trace constraints over \tau that final-state inspection cannot see. Under *Witness Grounding*, for every target field marked require_witness: true in the contract (such as a calculated GPA or a looked-up student ID), AURA verifies that the value written to SQLite or the filesystem appears in (or is a deterministic function of) the return_payload of a designated upstream read tool in \tau, and that the read tool's output did not violate a schema or sanity invariant (catching both **guessed values** and **silent tool errors**). Under *Temporal Ordering & Negative-Space Guards*, AURA verifies precedence rules a_i \prec a_j—for instance, that verify_student_eligibility was called and returned True *before* update_marks was invoked, and that no subsequent modification to the student record occurred *after* generate_report without regenerating the report.

3. Stage 4: The Three-Valued Kleene Logic (\mathcal{K}_3) Epistemic Arbiter and Diagnostic Ledger

To make PASS, FAIL, and UNCERTAIN mathematically rigorous rather than ad-hoc heuristics, assign every atomic predicate c_i \in \mathcal{P} \cup \mathcal{R} \cup \mathcal{T}_{\text{spec}} a truth value in Strong Kleene three-valued logic \mathcal{K}_3 = \{1 (\text{True}), \frac{1}{2} (\text{Un[span_196](start_span)[span_196](end_span)[span_199](start_span)[span_199](end_span)known}), 0 (\text{False})\}:

- v(c_i) = 1 iff the storage surface or trace channel required by c_i is marked OBSERVABLE in \Omega, obs_certainty == CONFI[span_37](start_span)[span_37](end_span)RMED, and the predicate holds.
- v(c_i) = 0 iff the surface or channel is OBSERVABLE (or a definite error receipt was returned) and the predicate is violated.
- v(c_i) = \frac{1}{2} iff the surface required by c_i is marked UNOBSERVABLE or READBACK_DENIED in \Omega, or the tool call required to materialize c_i returned obs_certainty == UNCONFIRMED (such as a timeout with unknown externalization, Anomaly A2).

The global workflow verdict V(\mathcal{C}, \Delta S, \tau, \Omega) is the strong Kleene conjunction across all predicates:

V = \min_{c_i \in \mathcal{P} \cup \mathcal{R} \cup \mathcal{T}_{\text{spec}}} v(c_i)

- **PASS (V = 1)**: Every positive postcondition, cardinality bound, closed-world residue check, and trace provenance rule evaluated to 1.
- **FAIL (V = 0)**: At least one predicate evaluated to 0. Notice a critical property of Kleene conjunction: \min(0, \frac{1}{2}) = 0. **If an external email tool timed out (\frac{1}{2}), AND the agent corrupted an unrelated student's marks in SQLite (0), the verdict is FAIL (0), never UNCERTAIN.** Known failure always dominates epistemic uncertainty.
- **UNCERTAIN (V = \frac{1}{2})**: Every observable positive postcondition, residue check, and provenance check evaluated to 1 (v(c_i) \ge \frac{1}{2} for all i), *and* at least one required check evaluated to \frac{1}{2} due to an unconfirmed external boundary or denied readback channel.

Alongside the top-level gate, AURA emits a JSON **Verification Evidence Record** containing a primary diagnostic sub-verdict (FAIL_INCOMPLETE, FAIL_VALUE_MISMATCH, FAIL_DUPLICATE_EFFECT, FAIL_RESIDUE_PERSISTENT, FAIL_RESIDUE_TRANSIENT, FAIL_PROVENANCE_UNGROUNDED, UNCERTAIN_UNCONFIRMED_EXTERNAL, UNCERTAIN_BLIND_SURFACE), the exact diff tuples in \Delta S_{\text{unmet}} and \Delta S_{\text{residue}}, and the offending step index t from \tau.

Feasibility Scope: 2–3 Day Laptop Pilot vs. 4–6 Week Research System

Architectural Feature	2–3 Day Laptop Pilot (Minimal AURA)	4–6 Week Research System (Improved AURA)
Environment & Storage	Single SQLite database (sqlite3) + local workspace folder + simulated Python tool wrappers.	Multi-database SQLite/PostgreSQL + local filesystem + mock HTTP external webhook server with

Architectural Feature	2–3 Day Laptop Pilot (Minimal AURA)	4–6 Week Research System (Improved AURA)
		configurable timeout and drop rates.
State Diffing (\Delta S)	Python script dumping table rows & file SHA-256/text before (S_0) and after (S_T) to compute exact JSON diffs.	Canonical relational diff engine + SQLite TEMP TRIGGER audit log capturing intermediate dirty writes and database trigger cascades.
Contract Schema (\mathcal{C})	Declarative JSON schema checking positive row/file values, expected_count, and closed-world \Delta S_{\text{residue}} = \emptyset.	Full Dual-Surface contract: JSON state assertions + cross-store variable bindings + AgentLTL-style witness grounding & temporal precedence rules.
Epistemic UNCERTAIN Handling	Simulated via a mock external notification tool that can return TIMEOUT_UNKNOWN and a table with simulated readback revocation.	Formal Observation Boundary Manifest (\Omega) implementing full Kleene \mathcal{K}_3 truth aggregation across multi-tool chains and orphaned compensations (A1–A4).
Agent / Simulator	Deterministic scripted workflow simulator with parameterized fault injection (zero LLM API cost; runs 500 trials in under 10 seconds).	Scripted fault simulator **plus** local open-weight LLMs (via Ollama/vLLM, such as Qwen or Llama models) or free-tier APIs to test real agent trajectories and recovery loops.

6. Novelty Opportunities: What Research Questions Remain Genuinely Open?

To publish an honest, rigorous paper, you must clearly separate what is already established from where you can make a genuine scientific contribution.

What You Must NOT Claim as Novel

First, **do not claim novelty for "evaluating AI agents by diffing initial and final database states rather than trusting tool calls."** AppWorld (2024), WorkBench (2024), τ-bench (2024), and Agent-Diff (Feb 2026) have already established this paradigm. Second, **do not claim novelty for "using a closed-world invariant over state diffs to catch unrequested side effects."** AppWorld ("collateral damage" unit tests), WorkBench ("side effects" metric), and Agent-Diff ("closed-world invariant") already define and measure this exact property. Third, **do not claim novelty for "precondition and postcondition runtime contracts for LLM agents."** AgentSpec (ICSE 2026), Agent Behavioral Contracts (Feb 2026), and EffectMatch (Sep 2026) already occupy that space.

Four Genuine Open Research Opportunities (As of October 2026)

1. **Opportunity 1: Closing the State-vs-Trace Blind Spot (Joint Delta-Trace Arbitration under Information Parity)** Current deterministic verifiers are split between **extensional state-diff checkers** (Agent-Diff, AppWorld) and **intensional trace monitors** (AgentLTL, Vigil). No prior study systematically characterizes the **non-overlapping failure modes** of these two paradigms on the same benchmark under strict information parity—showing how state-only verifiers fail on epistemic shortcuts, silent tool corruption, and transient dirty writes, while trace-only verifiers fail on silent persistent write drops (MedAgentBench ungrounded oracle), hidden database trigger cascades (EffectMatch), and cross-step state regressions.
2. **Opportunity 2: Formalizing Epistemic Abstention (UNCERTAIN) Under Partial Observability** Existing benchmark evaluators assume omniscient white-box access to the entire world state and force a binary PASS/FAIL output. In real deployments, agents interact with hybrid environments combining white-box local stores (SQLite, filesystems) and black-box external APIs (email gateways, payment webhooks) where outcomes can be unconfirmed (Trofimov & Novikov's Anomalies A1–A3). Formalizing a **Kleene \mathcal{K}_3 Epistemic Arbiter** and evaluating its **Selective Classification Risk-Coverage trade-off**—converting silent false accepts and false rejects into calibrated UNCERTAIN escalations as observation boundary legibility degrades, paralleling institutional abstention findings in arXiv:2609.03192—is a novel and timely contribution.
3. **Opportunity 3: Mutation-Audited Task Contracts ("Hardening Verifiers Against Silent False Accepts")** Abrich (Oct 2026, arXiv:2610.09142) and Bellibatlu et al. (Sep 2026, arXiv:2609.37315) demonstrated that shipped benchmark verifiers suffer from false accepts on duplicate non-idempotent writes (6 of 15 constructed effects in AppWorld) and out-of-scope field writes (21 of 23 selected cases in WorkArena). While Abrich notes that mutation testing of benchmark verifiers itself is now established, you can contribute an **Automated Contract-Hardening & Synthesis Loop** inside AURA that automatically generates fault-mutated trajectories (duplicate writes, partial writes, guessed witnesses, extra-column mutations) against draft JSON contracts, identifies surviving mutants, and automatically repairs the contract (by injecting missing cardinality bounds, frame constraints, and witness clauses) to guarantee a **100% Mutation Kill Rate** before deployment.
4. **Opportunity 4: Overcoming the "Verifier Tax" via Diagnostic-Guided Selective Rollback** A June/October 2026 systematic review (Toward Safe LLM Agents, arXiv:2608.14590) highlighted the **Verifier Tax**: in one \tau-Bench study of transactional tool-use agents, blocking 94% of unsafe actions via runtime guardrails left safe task completion below 5%, as agents found unsafe alternatives or failed to recover (the survey notes this finding currently rests on a single study and requires independent replication). Pairing AURA's fine-grained diagnostic sub-verdicts (FAIL_INCOMPLETE vs. FAIL[span_183](start_span)[span_183](end_span)[span_191](start_span)[span_191](end_span)_RESIDUE vs. FAIL_PROVENANCE_UNGROUNDED) with SQLite SAVEPOINT rollback (AgentRewind) allows you to independently test and resolve the Verifier Tax by measuring whether **diagnostic-guided selective rollback** restores safe task completion.

7. Experimental Plan: How Should We Validate Our

Claims Fairly?

To ensure your study survives critical peer review, your experimental design must eliminate the **information-asymmetry confound** and evaluate all methods across a balanced matrix of fault classes and observability regimes.

1. Enforcing Experimental Fairness: The Information-Parity Protocol

To demonstrate that AURA's gains come from its **verification architecture** rather than privileged data access, your experiment should evaluate **six distinct baselines**, where Baselines A2, B1, B2, B3, and C are all granted access to the exact same underlying storage state and contract rules:

1. **Approach A1 — Naive Execution Trust (Status-Quo Reference Only)**: Checks only that all called tools returned status == "ok" and the agent emitted a completion message. *(Report this only to quantify baseline deployment risk; explicitly state in your paper that comparing A1 to C is not information-matched).*
2. **Approach A2 — Information-Matched Agentic Self-Verification (Read-Back Prompting / Judge)**: Grant the agent (or a post-execution LLM-as-a-Judge / Agent-as-a-Judge) the **exact same readback queries** (SELECT * FROM ..., directory listings, file reads) that AURA uses, and explicitly prompt it to verify all requirements and check for side effects before outputting PASS, FAIL, or UNCERTAIN (mirroring the "doubt and refetch" baseline in RITE).
3. **Approach B1 — Step-Local Tool Postcondition Verification**: Immediately after each individual tool call a_t, a deterministic checker queries the SQLite table or file targeted by a_t to verify its local postcondition Q_t. However, B1 does *not* perform end-of-workflow global state diffing \Delta S = \text{Diff}(S_0, S_T) or cross-step provenance checking.
4. **Approach B2 — Trace-Only Formal Verification (AgentLTL / Vigil Proxy)**: Evaluates the complete execution trace \tau (tool names, arguments, return payloads, ordering, and witness grounding) against the contract, but never inspects the out-of-band storage snapshots S_0, S_T.
5. **Approach B3 — State-Diff-Only Verification (Agent-Diff / AppWorld Proxy)**: Computes the complete storage diff \Delta S = \text{Diff}(S_0, S_T) across SQLite and the filesystem, checking positive assertions and the closed-world residue invariant, but ignores the execution trace \tau and treats external boundary timeouts as binary pass/fail. *(Run B3 in two sub-modes: B3-Existential, which mimics AppWorld's record-count blindness, and B3-StrictCount, which mimics Agent-Diff).*
6. **Approach C — Full AURA (Dual-Surface Delta-Trace + Kleene \mathcal{K}_3 Boundary Arbiter)**: Combines \Delta S (with strict cardinality and closed-world residue), trace witness provenance \tau, SQLite intermediate audit logs, and the Kleene \mathcal{K}_3 observation boundary evaluator.

2. Realistic Multi-Step Task Scenarios (3 Domains, 15 Templates, 45+ Instantiations)

Following AppWorld (which instantiates 3 variants per scenario generator) and RITE (which pairs each unit across clean, silent-error, and explicit-error conditions), build 15 task templates across 3 domains (3 parameter variants each = 45 base tasks):

- **Domain 1: Academic Records & Compliance Workflow (Your Core Scenario)**: Creating and updating student profiles in students, computing weighted grades from a raw CSV or JSON transcript file into marks, enforcing academic probation rules in standing, writing a formatted Markdown or JSON report to /reports/student_<id>.md, and invoking a simulated notify_registrar_webhook() API while leaving all other students' rows untouched.
- **Domain 2: E-Commerce Order Modification & Refund Reconciliation (Inspired by τ-bench & RITE)**: Checking policy eligibility (24-hour window), cancelling an order in orders, restoring item stock in inventory, issuing a refund in payments, logging a receipt file, and ensuring no duplicate refund rows or modifications to other orders occur.
- **Domain 3: Enterprise IT Access Provisioning & Audit Logging (Inspired by WorkArena & EffectMatch)**: Verifying employee department clearance, updating role_bindings, generating an SSH or configuration artifact on disk, and handling a SQLite database trigger on role_bindings that silently inserts an unapproved external alert into security_notifications when privileged roles are modified without a valid ticket ID.

3. Controlled Failure-Injection Taxonomy (7 Literature-Grounded Fault Classes)

For each of the 45 base tasks, run a **Clean Gold Trajectory** (must yield PASS) and inject 7 distinct fault families (yielding 45 × 8 = 360 controlled evaluation episodes):

- **F1: Silent Tool No-Op / Partial Write** (MedAgentBench / tau2 defect class): Tool returns {"status": "success"}, but drops one required column update or skips writing the report file to disk.
- **F2: Silent Tool Data Corruption** (RITE defect class): An upstream read tool (such as get_raw_exam_scores) returns a schema-valid but corrupted numeric value (for example, 50 instead of 85, or an out-of-bounds default), which the agent propagates into SQLite and the report.
- **F3: Collateral Residue & Hidden Trigger Cascade** (EffectMatch / WorkBench defect class): The agent updates student_id = 101 correctly, but either modifies student_id = 102 due to an off-by-one WHERE clause, leaves a temporary dump file in /reports/, or fires an unapproved SQLite trigger into an audit or notification table.
- **F4: Duplicate Non-Idempotent Write** (Abrich 2026 / Anomaly A1 defect class): A simulated network hiccup causes the agent to call insert_student_record or issue_refund twice, creating two rows with valid field values.
- **F5: Epistemic Shortcut / Guessed Witness** (AgentLTL / Vigil defect class): The agent skips calling the required calculation or validation tool (or ignores its output) and writes the target value directly from a prompt hint or hardcoded fallback.
- **F6: Late Cross-Step Regression / Dirty Transient Write**: Step 2 updates marks correctly (passing local check B1), Step 3 generates the report, and Step 4 runs a cleanup query that accidentally resets marks in SQLite (breaking cross-store consistency between SQLite and the saved report), OR Step 2 corrupts an unrelated row and Step 4 reverts it after an external reader already observed it (Anomaly A5).
- **F7: Degraded / Blind Observation Boundary (UNCERTAIN Ground Truth)** (Trofimov Anomaly A2/A3): All local SQLite and filesystem changes succeed cleanly with zero residue, *but* either (a) notify_registrar_webhook() returns HTTP 504 Timeout (Outcome Unconfirme[span_460](start_span)[span_460](end_span)d) with no status lookup

endpoint, or (b) the verifier's read access to the standing table is revoked in \Omega.

4. Ground-Truth Generation, Scoring UNCERTAIN, Metrics, and Reproducibility

To handle genuine uncertainty without biasing the evaluation score, every episode i must be assigned two ground-truth labels determined deterministically by the simulator's fault injector:

1. *Ontological World Ground Truth $(Y_i^{\text{world[span_117](start_span)[span_117](end_span)[span_120](start_span)[span_120](end_span)}} \in \{\text{SUCCESS}, \text{FAILURE}\})$*: Whether the underlying task actually succeeded in the omniscient simulator.
2. *Epistemic Ground Truth under Observation Map Ω $(Y_i^{\text{epistemic}}(\Omega) \in \{\text{PASS}, \text{FAIL}, \text{UNCERTAIN}\})$*: The mathematically optimal decision given the information visible through Ω. Specifically, for Fault Class F7 (where the external webhook timed out without a status API, and in 50% of F7 runs the webhook secretly succeeded while in 50% it secretly dropped), any deterministic binary verifier (B2 or B3) that guesses PASS or FAIL will be wrong 50% of the time. Only UNCERTAIN matches $Y_i^{\text{epistemic}}(\Omega)$.

Across all baselines and ablations, report five primary metrics:

1. **False-Accept Rate (FAR / Silent Incorrect Success Rate)**: Fraction of failed runs ($Y^{\text{world}} = \text{FAILURE}$) falsely graded as PASS. *(In safety-critical verification, driving FAR to 0% is the primary objective).*
2. **False-Reject Rate (FRR)**: Fraction of clean runs ($Y^{\text{world}} = \text{SUCCESS}$ under full observability) falsely graded as FAIL.
3. **3-Class Epistemic Accuracy, Macro-F1, and Chance-Corrected Cohen's κ** against $Y^{\text{epistemic}}(\Omega)$.
4. **Selective Classification Risk-Coverage Curve**: Plot verification error rate on committed (PASS/FAIL) runs against the abstention rate (UNCERTAIN fraction) as boundary legibility degrades from 100% to 0% (following the protocol of arXiv:2609.03192).
5. **Runtime Overhead**: Wall-clock latency (in milliseconds) and memory footprint of snapshot diffing and trace checking per task.

Because variants generated from the same task template are correlated, do **not** treat correlated variants as independent coin flips. Report **95% Wilson score confidence intervals** or **cluster-bootstrapped confidence intervals** clustered at the task-template level (1,000 bootstrap resamples, following WorkArena and Agent-Diff), and use McNemar's test on paired binary outcomes. For reproducibility, pin all package versions in pyproject.toml (uv.lock), fix random seeds for fault injection, store every episode's (S_0, S_T, ΔS, τ, verdict) in JSONL artifacts, and provide a single CLI command (pytest / p[span_311](start_span)[span_311](end_span)ython run_experiments.py) that regenerates all tables and figures from scratch.

8. Three Development Options: Minimal, Improved, and Alternative

Depending on your timeline and ambition, here are three concrete development paths evaluated from first principles:

Dimension	Direction 1: Minimal AURA *(2–3 Day Laptop Pilot)*	Direction 2: Improved AURA *(4–6 Week Publishable Arbiter)*	Direction 3: Alternative Idea *(AURA-Recover: Diagnostic Rollback Runtime)*
Core Thesis	Post-execution state diffing (\Delta S) with a closed-world residue invariant catches silent write drops and collateral damage missed by tool-level checks.	**Joint State-Diff + Trace-Provenance Arbitration** with a **Kleene \mathcal{K}_3 Epistemic Boundary** and **Mutation-Audited Contracts** eliminates the blind spots of state-only and trace-only verifiers.	Verification without recovery causes the **Verifier Tax**. Pairing diagnostic state-diff receipts with **SQLite savepoint and file rollback** enables localized agent self-repair.
Architecture	Python + SQLite + local filesystem snapshotter (S_0, S_T), JSON contract parser, canonical diff engine, and closed-world residue check (\Delta S \setminus \Delta S_{\text{claimed}} = \emptyset).	Adds SQLite TEMP TRIGGER intermediate audit logs, typed tool-trace logger (\tau), witness-grounding checks, Kleene \mathcal{K}_3 boundary manifest (\Omega), and an Automated Contract Mutation Auditor.	Wraps an LLM agent (local Qwen/Llama or API) in a checkpointing harness (SQLite SAVEPOINT + shadow directory). When AURA returns FAIL_RESIDUE or FAIL_INCOMPLETE, it rolls back the bad step and feeds the diff receipt to the agent.
Novelty & Research Contribution	**Low scientific novelty** (reproduces Agent-Diff and AppWorld locally), **High educational & baseline value** as the foundation for Directions 2 or 3.	**Strong scientific novelty** in the evaluation and verification track (directly extends Abrich 2026, Agent-Diff, AgentLTL, and Trofimov 2026).	**Strong systems and agentic novelty** (bridges EffectMatch, AgentRewind, and Outcome Monitors to address the Verifier Tax).
Implementation Complexity	**Low (~400–600 lines of Python)**. Zero external dependencies beyond standard library, pydantic, and pytest.	**Moderate (~1,500–2,200 lines of Python)**. Runs 100% locally on a laptop CPU in seconds for scripted runs; optional local LLM for A2 baseline.	**Moderate-to-High (~2,000–2,500 lines of Python)**. Requires running live LLM rollouts (via Ollama or free/low-cost APIs) to measure multi-turn recovery rates.
Primary Limitations	Blind to guessed values (F5), transient dirty writes (F6), and contract bugs; UNCERTAIN is trivial without boundary	Requires explicit JSON contracts per task template; post-hoc arbitration detects failures after execution rather than repairing	Evaluation depends on the base LLM's ability to interpret diagnostic receipts (RITE shows models sometimes fail to integrate verification

Dimension	Direction 1: Minimal AURA *(2–3 Day Laptop Pilot)*	Direction 2: Improved AURA *(4–6 Week Publishable Arbiter)*	Direction 3: Alternative Idea *(AURA-Recover: Diagnostic Rollback Runtime)*
	modeling.	them mid-flight.	evidence).
Evaluation Strategy & Target Venue	Evaluate on 15 scripted tasks across Approaches A1, B1, B3; ideal for a GitHub portfolio artifact, course capstone, or student demo.	Evaluate across all 6 information-matched baselines and F1–F7 on 360 episodes + mutation kill-rate audit; target workshop (such as NeurIPS *Who Verifies the Agents?*) or SE/AI verification track.	Compare Continue vs. Binary Retry vs. AURA-Recover on task completion and residual side-effect rates; target AI systems or software engineering conferences (ICSE, VLDB, NeurIPS).

9. Final Recommendation: What Should You Build First, and What Should You Avoid?

Practical Value and Career Relevance for Student Researchers

Evaluating AURA against your career goals in AI engineering, backend development, and reliable agentic systems: **this is an outstanding project to build, provided you treat "Minimal AURA" (Direction 1) strictly as your 3-day stepping stone into "Improved AURA" (Direction 2).** In industry interviews for AI systems and backend engineering roles in 2026, almost every candidate has a GitHub repository with a basic prompt wrapper. Very few students can demonstrate a clean, zero-dependency Python runtime that implements **canonical database and filesystem state diffing, SQLite audit triggers, closed-world frame invariants, trace provenance verification, and mutation-tested contracts** backed by an information-matched ablation suite.

Step-by-Step Execution Playbook: What to Build First

1. **Days 1–3 (Build Direction 1 as Your Core Engine, With Two Crucial Upgrades)**: Build the Python + SQLite + filesystem state snapshotter (S_0, S_T) and canonical diff engine (\Delta S = \text{Diff}(S_0, S_T)). Instead of coding five separate modules for your five dimensions, implement the **three formal planes**: (1) Positive State & Cross-Store Assertions with mandatory expected_count, (2) the Closed-World Residue Set Difference (\Delta S_{\text{residue}} = \Delta S \setminus \Delta S_{\text{claimed}}), and (3) a basic Trace Witness check verifying that values written to SQLite or files appeared in an earlier tool read output in \tau. Build the Academic Records workflow simulator with deterministic fault injectors for F1–F7 and run your first 100 scripted episodes in under 5 seconds.
2. **Weeks 2–3 (Upgrade to Direction 2 for Genuine Research Novelty)**: Add the **Observation Boundary Manifest (\Omega)** and **Strong Kleene \mathcal{K}_3 truth table** so UNCERTAIN is triggered rigorously by simulated external webhook timeouts (Anomaly A2/A3) and revoked table read permissions. Implement the **Information-Parity Baselines** (B1 Step-Local, B2 Trace-Only, B3-Existential AppWorld-style, B3-StrictCount Agent-Diff-style, and A2 Agent Self-Verification using a free local LLM via Ollama). Then

build a 150-line **Contract Mutation Auditor** that automatically tests your JSON contracts against synthetic duplicate writes, partial writes, and out-of-scope edits to verify that your contracts achieve a 100% mutation kill rate.
3. **Week 4 (Optional Extension Toward Direction 3)**: Add a 100-line SQLite SAVEPOINT wrapper that uses AURA's structured diagnostic sub-verdicts (FAIL_INCOMPLETE vs. FAIL_RESIDUE) to automatically roll back contaminated state and let an LLM agent retry with the diff receipt.

Five Pitfalls You Must Strictly Avoid

1. **Do NOT rig the baseline comparison by starving the baselines of readback access.** Always include B2 (Trace-Only), B3 (State-Diff-Only), and A2 (Agent with Readback Tools) under strict information parity.
2. **Do NOT use an LLM inside AURA's core verification loop.** The moment AURA relies on an LLM judge to decide PASS or FAIL, you lose determinism, inherit judge bias and kappa deflation, and become vulnerable to prompt injection and reasoning-extension DoS. Keep AURA 100% symbolic and deterministic.
3. **Do NOT write existential state checks without cardinality bounds.** Always assert expected_count on inserts and updates so duplicate non-idempotent writes (Anomaly A1) are caught.
4. **Do NOT use UNCERTAIN as a catch-all for malformed outputs or code exceptions.** Reserve UNCERTAIN strictly for epistemic partial observability (\min_i v(c_i) = \frac{1}{2} under Kleene logic \mathcal{K}_3).
5. **Do NOT spend weeks building a custom parser for a new Domain-Specific Language (DSL).** Standard declarative JSON schemas validated via Python's pydantic are cleaner, faster to build, and easier for other researchers to adopt.

10. Research Questions: Precise, Testable Formulations

Anchor your paper and experimental repository around these three falsifiable research questions:

- **RQ1 (Complementary Blind Spots Under Strict Information Parity)**: *When holding sandbox observability and task specifications constant across verifiers, what proportion of injected multi-step agent failures—specifically silent tool no-ops (F1), silent tool data corruptions (F2), hidden trigger and collateral residues (F3), duplicate non-idempotent writes (F4), epistemic shortcuts (F5), and late cross-step regressions (F6)—are missed by single-surface verifiers (Trace-Only B2 and State-Diff-Only B3) and agentic readback self-verification (A2), but deterministically caught by Joint Delta-Trace Arbitration (AURA)?*
    - **Testable Hypothesis (H_1)**: State-Diff-Only verification (B3) will achieve 0% False-Accept Rate (FAR) on persistent state drops (F1) and persistent residue (F3) when cardinality bounds are enforced, but will suffer >80% FAR on epistemic shortcuts (F5) and transient reverted writes (F6); conversely, Trace-Only verification (B2) will catch 100% of epistemic shortcuts (F5) but suffer >80% FAR on silent persistent write drops (F1) and unapproved database trigger cascades (F3). Joint Delta-Trace Arbitration (AURA) will reduce FAR to 0% across F1–F6 with <15 ms verification overhead per episode.

- **RQ2 (Epistemic Calibration Under Degraded Observation Boundaries)**: *As the fraction of task effects crossing unconfirmed external boundaries or read-restricted storage surfaces increases from 0% to 50%, how effectively does Strong Kleene (\mathcal{K}_3) Epistemic Arbitration convert silent false accepts and false rejects into calibrated UNCERTAIN abstentions compared to binary verifiers?*
    - **Testable Hypothesis (H_2)**: Under degraded observation legibility (Fault Class F7), binary state and trace verifiers (B2, B3) will exhibit a monotonic linear increase in silent false accepts and false rejects, whereas AURA's Kleene \mathcal{K}_3 arbiter will maintain a **0% False-Accept Rate on all observable failure modes** (since \min(0, \frac{1}{2}) = 0) while tracking missing boundary evidence monotonically via UNCERTAIN verdicts.
- **RQ3 (Contract Adequacy via Automated Mutation Auditing)**: *To what extent do naive declarative state contracts—written without explicit record-multiplicity bounds, closed-world residue invariants, or witness provenance clauses—suffer from false accepts under automated trajectory mutation, and how much does enforcing these three contract primitives increase the Mutation Kill Rate?*
    - **Testable Hypothesis (H_3)**: Building on the audit methodology of Abrich (2026), baseline existential state contracts will falsely accept 100% of duplicated non-idempotent writes (F4), 100% of unwitnessed guesses (F5), and 100% of unmentioned table side effects (F3); adding strict cardinality bounds, closed-world diff accounting (\Delta S_{\text{residue}} = \emptyset), and trace witness clauses will raise the contract suite's Mutation Kill Rate to 100% across all seeded fault classes.

Works cited

1. From Local Correctness to Task Completion in Long-Horizon Agents, https://arxlens.theagents.company/paper/2610.19686 2. Runtime Validation of Persistent Outcomes in Agent Workflows - arXiv, https://arxiv.org/pdf/2609.31301 3. When Tool Calls Succeed but Workflows Fail: Anomalies at ... - arXiv, https://arxiv.org/pdf/2609.15397 4. SagaLLM: Context Management, Validation, and Transaction, https://www.researchgate.net/publication/389855859_SagaLLM_Context_Management_Validation_and_Transaction_Guarantees_for_Multi-Agent_LLM_Planning 5. What Coding Agents Leave Unsaid: Measuring Task-State Gaps, https://arxiv.org/abs/2610.20003?ref=agents-report.net 6. RITE: Evaluating End-to-End Resilience to Silent Tool Errors in E, https://arxiv.org/html/2610.27777 7. Silent Errors Halve Agent Success: 33.6% Noticed at Best | AIChieve, https://aichieve.net/isolation-isnt-detection/ 8. AppWorld: A Controllable World of Apps and People for ... - alphaXiv, https://www.alphaxiv.org/abs/2407.18901 9. A Controllable World of Apps and People for Benchmarking ... - ar5iv, https://ar5iv.labs.arxiv.org/html/2407.18901 10. WorkBench: a Benchmark Dataset for Agents in a Realistic ... - arXiv, https://arxiv.org/html/2405.00823v1 11. Agent-Diff: Benchmarking LLM Agents on Enterprise API Tasks via, https://www.alphaxiv.org/abs/2602.11224 12. Runtime Enforcement of Behavioral Specifications in AI Agent Skills, https://arxiv.org/html/2606.26524v1 13. Finding Blind Spots in AppWorld and WorkArena Task Verifiers - arXiv, https://arxiv.org/html/2610.09142v1 14. APIFlow-Bench: The Enterprise Survival Test for AI Agents, https://blog.postman.com/apiflow-bench/ 15. AgentLTL: A Trace-Verification Framework for Measuring, Enforcing, https://arxiv.org/html/2607.02599v1 16. When Tool Calls Succeed but Workflows Fail:Anomalies at the,

https://arxiv.org/html/2609.15397v1 17. Toward Safe LLM Agents: A Survey of Specification, Verification, https://arxiv.org/html/2608.14590v2 18. Runtime Compliance Verification for AI Agents - arXiv, https://arxiv.org/pdf/2606.19242 19. A Data-Centric Survey of Privacy in LLM Agents - arXiv, https://arxiv.org/html/2606.26627v1 20. Do Agent Benchmarks Do What They Say?An Executable-Contract, https://arxiv.org/html/2609.37315v1 21. Customizable Runtime Enforcement for Safe and Reliable LLM Agents, https://arxiv.org/pdf/2503.18666? 22. Pro2Guard: Proactive Runtime Enforcement of LLM Agent Safety via, https://arxiv.org/html/2508.00500v1 23. GoEx: A Runtime for Autonomous LLM Applications - Gorilla, https://gorilla.cs.berkeley.edu/blogs/10_gorilla_exec_engine.html 24. WebArena: A Realistic Web Environment for Building Autonomous, https://arxiv.org/html/2307.13854v4 25. Formal Modeling and Verification for Agent Workflow and Trajectory, https://arxiv.org/html/2606.06523v3 26. arXiv:2408.04682v1 [cs.CL] 8 Aug 2024, https://arxiv.org/pdf/2408.04682? 27. GuardianAgentBench: Where Agents Fail and How to Guard Them, https://www.alphaxiv.org/abs/2607.20982 28. Your AI Agent Is Modeling Your Hidden Tests: Audit the Grader It, https://medium.com/@Micheal-Lanham/your-ai-agent-is-modeling-your-hidden-tests-audit-the-grader-it-imagines-ff44d2c7f37b 29. Agent Behavioral Contracts: Formal Specification and Runtime, https://arxiv.org/html/2602.22302v1 30. Stop Treating LLM Monitors as Controls. Gate the Tool Call Instead, https://medium.com/@sattyamjain96/stop-treating-llm-monitors-as-controls-gate-the-tool-call-instead-61f47126cb75 31. AgentRewind: Giving Long-Horizon LLM Agents an Undo Button, https://www.zhongzhuzhou.org/blog/2026-08-17-agentrewind-technical-review-en/ 32. Benchmarking LLM Agents on Enterprise API Tasks via Code ... - arXiv, https://arxiv.org/html/2602.11224v3 33. AppWorld Benchmark Tasks - Emergent Mind, https://www.emergentmind.com/topics/appworld-benchmark-tasks 34. (PDF) Finding Blind Spots in AppWorld and WorkArena Task Verifiers, https://www.researchgate.net/publication/415375377_Finding_Blind_Spots_in_AppWorld_and_WorkArena_Task_Verifiers 35. Causal Past Logic for Runtime Verification of Distributed LLM Agent, https://arxiv.org/html/2605.20923v2 36. AgentLTL - arXiv, https://arxiv.org/pdf/2607.02599 37. An open source guardrail system for building secure AI agents - arXiv, https://arxiv.org/html/2505.03574v1 38. Tool Call Succeeds, Workflow Fails: External-Effect Anomalies at the, https://www.bloss0m.com/en/paper-reading/49-tool-calls-workflows-fail/ 39. A Validity Audit of Tool-Calling Evaluation - arXiv, https://arxiv.org/pdf/2607.02577 40. SagaLLM: Context Management, Validation, and Transaction ... - arXiv, https://arxiv.org/html/2503.11951v3 41. Durable Execution Is Not a Transaction: The Missing State in Agent, https://giniloh.com/durable-execution-not-a-transaction/ 42. Strategic Verification for Long-Running LLM Agents[v1] | Preprints.org, https://www.preprints.org/manuscript/202608.2057 43. AgentRewind: Recoverable Execution for Long-Horizon LLM Agents, https://arxiv.org/html/2608.14380v1 44. WorkBench Revisited: Workplace Agents Two Years On - arXiv, https://arxiv.org/html/2606.13715v1 45. Dhruv1000/qwen35-logra-ifbench at main - Hugging Face, https://huggingface.co/Dhruv1000/qwen35-logra-ifbench/blob/main/docs/research/m6/METHODOLOGY_PRIOR_ART.md 46. [2407.18901] AppWorld: A Controllable World of Apps and People, https://arxiv.org/abs/2407.18901 47. AppWorld: A Controllable World of Apps and People for ... - GitHub, https://github.com/StonyBrookNLP/appworld 48. A Controllable World of Apps and People for Benchmarking ... - arXiv, https://arxiv.org/html/2407.18901 49. AppWorld: A Controllable World of Apps and People for, https://aclanthology.org/2024.acl-long.850.pdf 50. Agent Diff | Deterministic Evaluation, https://www.agentdiff.dev/ 51. The Agent Safety Gate

Moves to Verified State | Giniloh, https://giniloh.com/agent-safety-gate-moves-to-verified-state/ 52. SagaLLM: Context Management, Validation, and Transaction, https://arxiv.org/html/2503.11951v1 53. [2609.15397] When Tool Calls Succeed but Workflows Fail - arXiv, https://arxiv.org/abs/2609.15397 54. Outcome Monitors: Recovery Affordances for Silent Tool Failures, https://arxiv.org/pdf/2608.19303 55. τ-bench: A Benchmark for Tool-Agent-User Interaction in ... - arXiv, https://arxiv.org/pdf/2406.12045 56. PRIOR_ART.md - kimjune01/generative-tau-bench - GitHub, https://github.com/kimjune01/generative-tau-bench/blob/main/docs/PRIOR_ART.md 57. Experimental Localisation of Behavioural Properties in an Agent, https://arxiv.org/html/2609.03192v1 58. ICML Poster Agent-as-a-Judge: Evaluate Agents with Agents, https://icml.cc/virtual/2025/poster/45485 59. WorkArena: How Capable are Web Agents at Solving Common, https://raw.githubusercontent.com/mlresearch/v235/main/assets/drouin24a/drouin24a.pdf 60. LLM-as-a-Judge Validity: What 541,000 Decisions Reveal, https://kili-technology.com/blog/data-story-who-judges-the-judge-llm-as-a-judge 61. (PDF) AgentSpec: Customizable Runtime Enforcement for Safe and, https://www.researchgate.net/publication/390142942_AgentSpec_Customizable_Runtime_Enforcement_for_Safe_and_Reliable_LLM_Agents 62. Planarian: Managing Agent State with Statepoints - arXiv, https://arxiv.org/html/2609.35366v1 63. DeFiFlowBench: Benchmarking and Improving Safe Executability in, https://arxiv.org/html/2609.11504v1 64. τ-Bench: A Benchmark for Tool-Agent-User Interaction in Real-World, https://github.com/sierra-research/tau2-bench.git?rev=b209e7557e42aa3ae37ededeaa71a6d3234d362b 65. Agent Behavioral Contracts II: Certifying Compositional Reliability, https://arxiv.org/pdf/2608.12895 66. Denial-of-Service Attacks on LLM-Based Agent Guardrails - arXiv, https://arxiv.org/html/2606.14517v1 67. The Rise of Agent-as-a-Judge Evaluation for LLMs - arXiv, https://arxiv.org/html/2508.02994v1